Republic of Tunisia
Ministry of Higher Education and Scientific Research
University of Tunis El Manar
National Engineering School of Tunis

Information and Communication Technologies Department

End-of-Year Project I

AI-Enhanced IDS Integration for Firewalls

Prepared by:
KHAMMAR Amir

Class:
1st Year Telecommunications

Supervised by:
Mr. Faouzi BAHLOUL

Academic Year: 2025 – 2026

Signature

supervisor : M. Faouzi Bahloul

Acknowledgements

It is with great pleasure that I dedicate these lines to express my sincere gratitude to all those who contributed to the completion of this report.
I would first like to express my sincere gratitude to my supervisor Mr.Faouzi BAHLOUL, for his guidance, encouragement and support throughout the progress of this project.
I also wish to extend my heartfelt thanks to Mme Rym Regaieg, who provided me technical assistance and guided me at the practical implementation of the project .
Finally, I am grateful to my professors, colleagues and especially my family for their moral support .

Abstract

This final year project focuses on the integration of an intrusion detection system (IDS) with a firewall to improve network security. OPNsense was used as the firewall platform and Suricata as the IDS/IPS engine within a virtual lab simulating attack scenarios.
The objectives were to configure and deploy OPNsense, integrate Suricata, simulate attacks and analyze logs.
A Random Forest model was then applied to enhance detection and reduce false positive.

Keywords: Network Security, Firewall, IDS, IPS, OPNsense, Suricata,
Machine Learning, Random Forest

Contents

List of Figures **5**

List of Tables **6**

1 Fundamentals of Network Security **10**
1.1 Introduction to Network Security . . . 10
1.1.1 Definition and Scope . . . 10
1.1.2 CIA Triad . . . 10
1.2 Common Network and Attack Vectors . . . 11
1.2.1 Overview . . . 11
1.2.2 Types of Attacks . . . 11
1.2.3 Impact . . . 11
1.3 Firewall Technologies . . . 11
1.3.1 Definition . . . 11
1.3.2 How a Firewall Works . . . 12
1.3.3 Stateful Inspection and Connection Tracking . . . 15
1.3.4 Firewalls Generations . . . 15
1.3.5 Historical Overview . . . 15
1.3.6 Limitations Of Firewalls . . . 16
1.4 Intrusion Detection System . . . 17
1.4.1 Definition and Role . . . 17
1.4.2 IDS vs IPS . . . 17
1.4.3 Deep Packet Inspection (DPI) . . . 17
1.4.4 Types of IDS . . . 18
1.4.5 Placement in Network Architecture . . . 19
1.4.6 Limitation of IDS/IPS . . . 19
1.5 Artificial Intelligence in Cybersecurity . . . 20
1.5.1 Machine Learning Approach for IDS . . . 20
1.6 Conclusion . . . 21

2 System Implementation and Lab Design **22**
2.1 Introduction . . . 22
2.2 Virtualization Environment . . . 22
2.2.1 Initial Attempt . . . 22
2.2.2 Lab Architecture Overview . . . 22
2.3 Network Configuration . . . 23
2.3.1 Network Adapters and Topology . . . 23
2.3.2 Connectivity Verification . . . 23
2.3.3 Challenges and Solutions . . . 24

2.4 OPNsense Firewall Configuration 25
2.4.1 Introduction 25
2.4.2 Interface Configuration and Default Behavior 25
2.4.3 Web Application Attack Without IDS 27
2.5 Suricata IDS Configuration 27
2.5.1 Introduction 27
2.5.2 Installation and Rule Sets 28
2.5.3 Challenges and Solutions 28
2.6 Attack Simulation 28
2.6.1 Attack Scenarios and Methodology 28
2.7 Dataset Preparation 29
2.7.1 Log Extraction 29
2.7.2 Feature Selection and Preprocessing 29
2.8 Conclusion 29

List of Figures

1.1 The CIA Triad 10
1.2 Firewall deployment topology 12
1.3 How packet travels through Netfilter 14
1.4 Basic architecture and placement of NIDS and HIDS 18
1.5 IDS placement in network architecture 19

2.1 OPNsense CLI with interfaces assignments 23
2.2 OPNsense successfully pinging Ubuntu (192.168.1.16) on the internal LAN 24
2.3 OPNsense successfully pinging Kali Linux (192.168.1.20) on the internal LAN 24
2.4 Bidirectional connectivity confirmed between Kali Linux and OPNsense WAN 25
2.5 OPNsense web GUI accessed from Ubuntu 25
2.6 OPNsense Live View showing blocked ICMP traffic 26
2.7 Nmap scan showing all ports filtered 27
2.8 Nmap scan report after NAT port forwarding with Pass association 27
2.9 XSS Payload successfully executed without IDS detection 27
2.10 Custom Rules written for each attack 28
2.11 Suricata alert dashboard 29

List of Tables

1.1 Key Milestones in Firewall Technology Development 16
1.2 Key differences between IDS and IPS 17

List of Abbreviations

ASA	Adaptive Security Appliances
CLI	Command Line Interface
DDoS	Distributed Denial of Services
DHCP	Dynamic Host Configuration Protocol
DNS	Domain Name System
DoS	Denial of Service
GUI	Graphic User Interface
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
IPsec	Internet Protocol Secuity
IPv4	Internet Protocol Version 4
LAN	Local Area Network
ML	Machine Learning
NAT	Network Address Translation
NIC	Network Interface Card
OS	Operating System
SCP	Secure Copy Protocol
SIEM	Security Information and Event Management
SSL	Secure Sockets Layer
TLS	Transport Layer Security
WAN	Wide Area Network

General Introduction

NETWORK SECURITY is continually becoming an area of tremendous focus for companies of all sizes, due the increasing number of cyber attacks . Firewalls and Intrusion Detection System (IDS) are essential tools in protecting infrastructure;however, conventional solutions frequently demonstrate insufficient adaptability when confronted with rapidly evolving threat landscapes [1]. The fundamental challenge addressed here resides in the inherent limitation of conventional firewalls, which can filter traffic but struggle to detect complex intrusions. IDS solutions provide deeper analysis, yet they may generate false positives and requires a smarter approach to improve accuracy.

In this context, the objective of this project is to design and implement a secure network environment by integrating OPNsense as a firewall with Suricata as an IDS/IPS engine. A virtual lab was developed to simulate attack scenarios, collect logs, and apply a Random Forest machine learning model to reduce false positives.
the report is structured as follows: the first chapter presents the theoretical background and related work; the second chapter describe the implementation and configuration; the third one discusses the experimental results; and finally, the conclusion summarizes key findings and outlines perspective research directions.

Chapter 1

Fundamentals of Network Security

1.1 Introduction to Network Security

1.1.1 Definition and Scope

Network security includes the tools, techniques, and practices used to protect computer networks and the critical data they transmit. Its primary purpose is to keep networks running safely by preventing unwanted access, detecting vulnerabilities, and responding effectively to potential attacks. [2].

1.1.2 CIA Triad

In the context of networking , the term security refers to the measures and practices designed to safeguard data, applications, devices and system from unauthorized access, misuse, or damage. its primary objective is to preserve the three fundamentals principles of information protection, commonly known as the CIA triad: [3]

- **Confidentiality** ensures that sensitive information is accessible only to authorized users, typically through encryption and access control.
- **Integrity** guarantees that data remain accurate and unalerted, using mechanisms such as hashing and digital signatures.
- **Availability** ensures that systems and data are accessible when needed, supported by redundancy, backups, and disaster recovery plans.

Principle	Description
Confidentiality	Only authorized users can view the data.
Integrity	The data remains accurate and unaltered.
Availability	Network resources are accessible when needed.

Figure 1.1: The CIA Triad

1.2 Common Network and Attack Vectors

1.2.1 Overview

Modern network infrastructure face a diverse array of threat that continue to evolve targeting the CIA Triad . Attackers exploit vulnerabilities in network, applications and humain behavior to compromise security. [4]

1.2.2 Types of Attacks

- **Malware:** Malicious software such as viruses, worms, Trojans, and ransomware designed to infiltrate and damage systems. [5]
- **Phishing:** social engineering attacks that trick users into revealing sensitive information through fraudulent emails or websites.
- **Denial-of-Service(Dos/DDos):** Flooding a system or network with traffic to make it unavailable to legitimate users. [6]
- **SQL Injection and XSS:** Code injection technique used to manipulate database or compromise web applications.
- **SYN flood attack:** a Type pf DDos that overwhelms a server by sending many initial requests(SYN packets) and never complete the TCP three way handshake. [7]

1.2.3 Impact

These attacks can lead to data breaches, financial losses, reputational damage and disruption of critical services.
Understanding them is essential for security.

1.3 Firewall Technologies

1.3.1 Definition

A firewall is a security system that monitors and controls all network traffic flowing between a trusted internal network and an untrusted external network such as the Internet. Its role is to enforce a set of administrator-defined rules, deciding for each packet whether it should be allowed to pass, blocked silently or rejected with a notification. [8]

Beyond its basic role, a firewall should not be viewed only as a filtering tool, but rather as a control system embedded within the network infrastructure. It does not simply sit beside the network it sits **in the middle of it**, meaning all data and packets must pass through it.

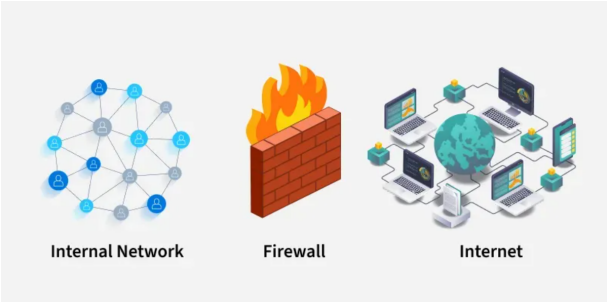


Figure 1.2: Firewall deployment topology

Firewalls can be deployed in several forms:

- **Hardware appliances:** Dedicated devices optimized for high-speed traffic processing (e.g, Cisco ASA).
- **Software-based firewalls:** implemented within operating system (e.g, `Netfilter` in Linux or `pf` in FreeBSD).
- **Virtual firewalls:** software instances running inside virtual machines, suitable for cloud environments.
- **Cloud-based firewalls:** Firewalls-as-a-Services (FWaaS) managed entirely in the cloud.

In this project, OPNsense is used as a software firewall running on a virtual machine, which belongs to the virtual firewall category.

1.3.2 How a Firewall Works

A firewall acts as the first line of defense in network security architecture. And understanding it requires looking at what happens inside the operating system when a packet arrives.

Core Functions

The Firewall performs three fundamental tasks :

- **Monitoring:** Watches all incoming and outgoing netwok traffic continuously.
- **Filtering:** Decides which traffic to allow or to block.
- **Protecting:** Prevents unauthorized access and stops threats from entering the network.

All data entering or leaving must first pass through it. No connection can bypass this checkpoint.

Network Adresses Translation(NAT)

NAT is a function almost always performed by firewalls. It changes IP addresses in packets to serve two purposes:

- **Source NAT (SNAT):** is a networking technique that modifies the source IP address in a packet's header and allow multiple internal devices to share one public IP(the firewall's public address)
- **Destination NAT (DNAT):** also called port forwarding. Incoming packets addressed to a specific public port are redirected to an internal server. This allows hosting a service (e.g, a web server) without exposing the internal IP directly.

NAT also provides an implicit layer of security by hiding the internal network structure from the outside world.

Virtual Private Network (VPN) Integration

Many modern firewalls, including OPNsense, integrate VPN capabilities to create encrypted tunnels over public networks:

1. **IPsec:** a network-layer protocol suite that encrypts entire IP packets, commonly used for site-to-site connection between offices.
2. **OpenVPN / SSL-TLS VPN:** application-layer tunnels allowing remote users to connect securely from anywhere, using standard HTTPS ports.

Packet Traversal and Kernel-level Interception

When a data packet enters a machine through the network card (NIC), it does not immediately reach any application. Instead, it travels through the operating system's **network stack** which is a series of layers inside the kernel that handle communication. The firewall inserts itself into this stack using **kernel hooks** which are predefined interception points where the firewall registers callback functions.
When a packet reaches one of these points, the kernel pauses its normal processing and hands the packet to the firewall for inspection. This interception happens in **kernel space** - the protected core of the OS so no user can bypass it.

In Linux-based and BSD-based systems (which OPNsense is build upon), this mechanism is provided by the **Netfilter** and **pf** framework. [9] Netfilter defines five hook points along the packet s path.

1. `PRE_ROUTING` - the packet has just arrived. The firewall sees it before any routing decision is made. Destination NAT (redirecting packets to internal servers) is applied here.
2. `LOCAL_IN` - the packet is destined for the firewall machine itself (e.g, access to the OPNsense Web GUI).
3. `FORWARD` - the packet is being relayed from one interface to another (e.g, from the Internet to the internal LAN). **This is the primary filtering point** where most firewall rules are evaluated.

4. `LOCAL_OUT` - the packet was generated by the firewall machine itself and leaving the system.

5. `POST_ROUTING` - the packet is about to leave the machine. Source NAT is applied here.

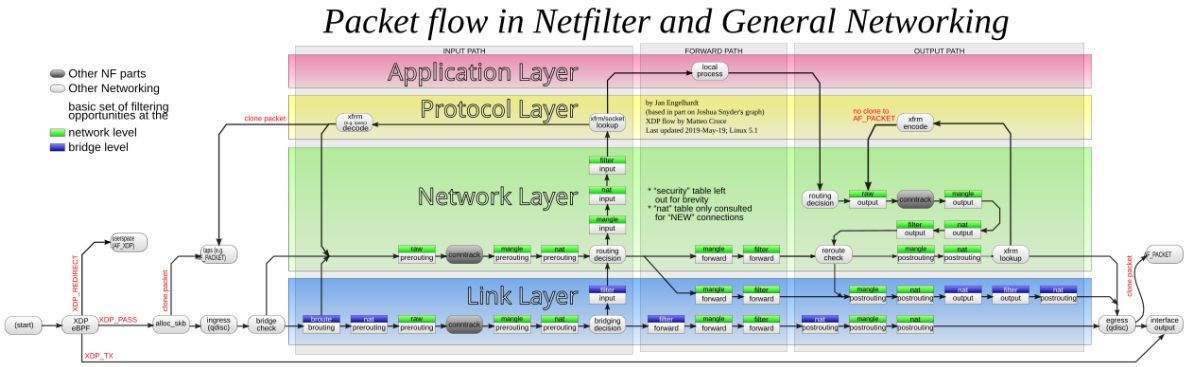


Figure 1.3: How packet travels through Netfilter

In each hook point, the firewall makes one of three decisions for each packet:

- **Accept:** Allow the traffic to pass.
- **Reject:** Block the traffic and send an error message back.
- **Drop:** Block the traffic silently without notification.

Rule Evaluation Engine

When a packet reaches the `FORWARD` hook, the firewall compares it against a list of predefined rules. Each rule specifies a set of conditions and an action to apply if those conditions are met. Typical conditions include:

- Source IP adress
- Destination IP adress
- Port number
- Protocol type (TCP, UDP, ICMP)
- Connection state (new, established, or invalid)

It processes rules in order from top to bottom. Once a packet matches a rule, the firewall applies that action immediately.

Default Policy: A firewall can apply a default policy(accept, reject, or drop). For example if no rule exists for SSH, the default applies. To prevent unauthorized access, it is best set to drop or reject. [10]

1.3.3 Stateful Inspection and Connection Tracking

Early firewalls evaluated each packet independently, with no memory of previous packets. Modern firewalls, including OPNsense, use **stateful inspection**: they maintain a *connection tracking table* the state table that records information about every active communication session.
This makes stateful inspection far more effective than simple rule matching alone. Each entry in this table stores:

- The source and destination IP addresses and ports
- The protocol being used
- The current state of the connection (new, established, or closing)
- For TCP: sequence number to verify packet legitimacy

1.3.4 Firewalls Generations

Firewall technology has evolved through distinct generations, each addressing the limitation of the previous one (see Table) . [10]

- **Packet filtering (1st generation):** Operating at OSI layers 3-4, the simplest form. Each packet is inspected individually based only on its headers (IP address, port, protocol). There is no memory of previous packets. High performance but vulnerable to spoofing and fragmented attacks.
- **Stateful inspection (2nd generation):** Introduced connection tracking. The firewall remembers the state of ongoig sessions and can detect spoofed packets.
- **Application proxy (3rd generation):** The firewall acts as an intermediary. It fully terminates the client's connection and connects to the destinationon behalf of the client. This allows inspection of the application-layer content (e.g, HTTP requests).
- **Next-Generation Firewall (NGFW, 4th generation):** Combines all previous capabilities with additional features such as Deep Packet Inspection (DPI), application identification regardless of port number, integrated Intrusion Prevention System and URL filtering. OPNsense with Suricata approach this category.

1.3.5 Historical Overview

Firewall technology emerged in the late 1980s as a direct response of the first major Internet security incidents, including the Morris Worm of 1988, which demonstrated how quickly malicious code could propagate across networks.

Table 1.1: Key Milestones in Firewall Technology Development

Year	Milestone
1988	Jeff Mogul at Digital Equipment Corporation (DEC) develops the first packet-filtering concepts, inspecting packets based on header fields.
1989–1991	AT&T Bell Labs introduces the circuit-level gateway, which validates connections without re-inspecting every packet.
1992	Marcus Ranum creates SEAL (Secure External Access Link), the first commercial application-layer firewall using proxy technology.
1994	Check Point releases FireWall-1, the first commercial stateful inspection product, establishing the dominant model for the next two decades.
2004	Gartner introduces the term "Next-Generation Firewall" to describe platforms combining DPI, IPS, and application awareness.
2018–Present	Artificial intelligence and machine learning are progressively integrated into firewall platforms to enable behavioral analysis and adaptive threat detection.

1.3.6 Limitations Of Firewalls

Despite their important role in network defense, firewalls have inherent limitations that no rule set can fully overcome. [11] A concrete example is `CVE-2024-20353`, a critical vulnerability discovered in 2024 affecting Cisco ASA and Firepower Threat Defense (FTD). This vulnerability was actively exploited in the **ArcaneDoor** espionage campaign targeting government networks worldwide.

- **Encrypted traffic** protocols such as TLS/SSL prevent firewalls from inspecting packet content for malicious payloads. Decrypting traffic inline is possible but introduces significant computational overhead and latency. [11]
- **Insider Threats:** trusted individuals misuse their access, since firewalls typically operate on predefined external-internal traffic rule, they lack the ability to monitor suspicious activities from internal users.
- **State table exhaustion:** in a SYN flood attack, the attacker send a large number of incomplete TCP connection requests, filling the state table untill the firewall can no longer track legitimate connections.
- **Evasion through permitted ports:** attackers frequently hide malicious traffic inside protocols that are always open, such as HTTP on port 80. A firewall cannot distinguish a legitimate web request from an attack disguised as one.
- **Zero-day exploits:** attacks that exploit unknown vulnerabilities have no existing signature, so rule-based filtering cannot block them.

These limitations show that a firewall alone is not sufficient. A deeper level of traffic analysis is required, one that can inspect packet content, detect behavior and respond to

threats that rules cannot anticipate. This is the role of Intrusion Detection and Prevention Systems.

1.4 Intrusion Detection System

1.4.1 Definition and Role

As established in the previous section, firewalls enforeces access control based on predefined rules but cannot inspect the content of permitted traffic.
An **Intrusion Detection System (IDS)** addresses this gap by monitoring network traffic and analyzing it for signs of malicious activity or policy violations. When a threat is detected, the IDS generate an alert for the administrator to investigate. It is also designed to document and log events for future analysis. [12]
An IDS is a **passive** system that observes and reports, but does not interfere with traffic.
An **Intrusion Prevention System (IPS)** extends this by acting inline in the traffic path, allowing it to block or drop malicious packets in real time. [13]

1.4.2 IDS vs IPS

While both systems aim to protect the network,their operating mode differs fundamentally.

Table 1.2: Key differences between IDS and IPS

Feature	**IDS**	**IPS**
Functionality	Detects and alerts on malicious activities	Detects, alerts, and blocks malicious activities in real time
Action	Does not take action, only provides alerts	Actively blocks or neutralizes threats to prevent damage
Response	Passive monitoring; alerts after the fact	Real-time, active prevention of threats
Use case	Monitoring and alerting about potential threats	Preventing breaches in real time

1.4.3 Deep Packet Inspection (DPI)

The filtering mechanism analyze packet **headers** only. They can determine where a packet comes from and where it is going, but cannot examine what it contains.
Deep Packet Inspection (DPI) closes this gap by going beyond header analysis to inspect the actual **payload** of each packet. DPI allows security systems to identify and block specific types of traffic such as malicious code or prohibited content that would otherwise pass undetected. [8]

From Header Analysis to Payload Inspection

Since network data is fragmented across many small packets, the DPI engine first reassembles them into the original data stream. It then identifies the application protocol in use - HTTP, DNS, FTP, TLS, and so on.
DPI engines rely on two detection strategies:

- **Signature-based detection:** the payload is compared against a regularly updated database of known attack patterns. Fast and reliable for known threats, but unable to detect new or unknown attacks.
- **Anomaly-based detection:** the engine learns a baseline of normal traffic behavior and flags significant deviations. Can detect previously unseen attacks, but tends to generate more false positives. [12]

1.4.4 Types of IDS

IDS solutions are classified based on where they are deployed and what they monitor. [14]

- **Network Intrusion Detection System (NIDS):** deployed at strategic points in the network to monitor traffic across all connected devices. It analyze packets flowing through the subnet and raises alerts when suspicious patterns are detected. This is the approach used by Suricata in this project.
- **Host Intrusion Detecion System (HIDS):** installed on individual machines. It monitors packets on that device only and can also track file system changes and system calls to detect local compromise.
- **Hybrid IDS:** combines NIDS and HIDS to provide a more complete view of both network-level and host-level activity. [14]

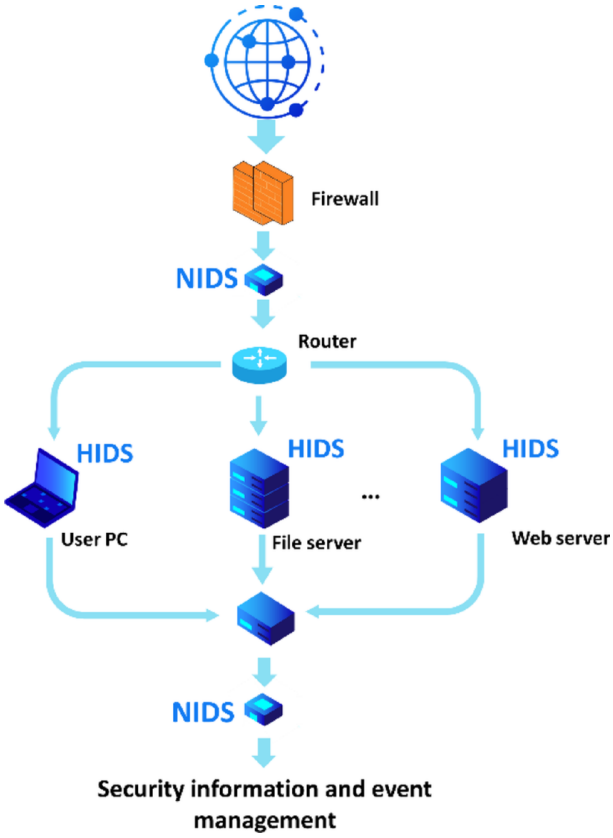


Figure 1.4: Basic architecture and placement of NIDS and HIDS [12]

1.4.5 Placement in Network Architecture

The position of an IDS within the network determines what traffic it can see and how effectively it detects threats. [14] Three common placement strategies exist:

- **Behind the firewall:** the IDS is placed on the internal network, after the firewall. It monitors only traffic that has already passed through the firewall rules, focusing on threats that actually entered the network. This reduces false positives and allows faster detection of successful intrusions. This is the placement used in our project.
- **In front of the firewall:** The IDS sees all incoming internet traffic before any filtering. Useful for mesuring the total volume of attack attempts, but generates significantly more alerts.
- **Inside the internal network:** The IDS monitors lateral traffic between internal machines, helping detect insider threats or an attacker who has already bypassed the perimeter and is moving within the network.

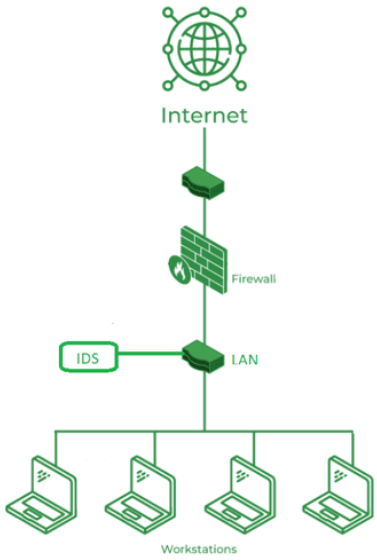


Figure 1.5: IDS placement in network architecture
[12]

1.4.6 Limitation of IDS/IPS

Despite their value, IDS and IPS solutions have well-known limitations that motivate the use of machine learning to improve their accuracy. [15]

- **False positives:** legitimate traffic incorrectly flagged as malicious, generating unnecessary alerts and consuming resources and analyst time. High false positive rates can overwhelm security team. This is the primary problem addressed in this project through a Random Forest classifier.
- **False negatives:** real attacks missed entirely, often because attackers use evasion techniques such as packet fragmentation, encoding, or protocol tunneling to hide malicious traffic from signature-based detection.

- **Signature dependency:** signature-based detection is blind to zero-day attacks and novel malware with no existing signature in the database. Regular updates are required . [16]

These limitations motivate the integration of machine learning into intrusion detection, which is the subject of the following section.

1.5 Artificial Intelligence in Cybersecurity

The growing sophistication of cyberattacks has made traditional rule-based security tools increasingly insufficient on their own. AI and ML in particular, have emerged as essential components of modern cybersecurity strategies. Traditional solutions such as firewalls and antivirus software are maintained by a limited number of security professionals and process data, limiting their ability to respond to today s fast evolving threats. [17]

1.5.1 Machine Learning Approach for IDS

ML methods applied to intrusion detection are generally divided into two main categories. [18]

- **Supervised Learning:** The model is trained on a labeled dataset containing both normal and malicious traffic samples. It learns to classify new traffic based on patterns found in the training data. This approach produces reliable results for known attacks types and achieves low false positives rates, but struggles to detect zero-day attacks with no prior signature.

- **Unsupervised learning:** The model learns a baseline of normal behavior without labeled data, flagging significant deviations as suspicious. This is better suited for detecting unknown attacks but typically generates a higher number of false positives.

Among supervised methods,**ensemble learning** approach have shown consistently strong performance in IDS tasks. Many current IDS solutions rely on rule-based systems that have limitations in detecting novel intrusions and require time-consuming manual updates. Applying data mining algorithms such as Random Forest to automate pattern learning offers a more scalable and accurate alternative. [1]

Random Forest for Intrusion Detection `Random Forest` builds a large number of decision trees during training, each constructed on a random subset of the data and a random subset of features. When classifying a new sample, every tree votes independently and the final decision is determined by majority vote. It supports both misuse detection and anomaly detection. [1]
This approach offers three advantages for IDS:

- **High accuracy:** combining multiple trees reduces errors and improves classification precision, especially on imbalanced datasets.

- **Resistance to overfitting:** the randomness introduced during training allows the model to generalize well to unseen traffic.

- **False positive reduction:** by learning patterns of both normal and malicious traffic, it distinguishes real threats from benign anomalies. [12]

Feature Engineering and Preprocessing Before any ML model can be trained, raw network traffic data must be transformed into a structured format that algorithm can process. This step is critical to the quality of the final model, as ML results are directly affected by the quality and relevance of the input data. [18]

1.6 Conclusion

Network security relies on multiple complementary layers of defense, firewalls provide essential perimeter control but cannot inspect traffic content. IDS/IPS systems fill this gap but still suffer from false positives that reduce their reliability. This project responds to these limitations by combining **OPNsense** as the firewall, **Suricata** as the IDS/IPS engine performing deep packet inspection, and a **Random Forest** classifier applied to Suricata's alert logs to reduce false positives and improve detection accuracy. The implementation of this architecture is presented in the following chapter.

Chapter 2

System Implementation and Lab Design

2.1 Introduction

This chapter presents the practical implementation of the security lab designed to demonstrate the limitations of firewalls and the integration of IDS/IPS system. The lab was build using virtualization and consists of three machines interconnected through OPNsense, which acts as the central firewall and security gateway.

2.2 Virtualization Environment

2.2.1 Initial Attempt

The project initially used VMware as the virtualization platform. However, kernel rolling issues and module incompatibilities after system updates caused instability and prevented the virtual machines from starting correctly. To resolve these issues and optimize resource usage given the available hardware `( 8 GB RAM)`, the environment was migrated to VirtualBox which provided better stability and compatibility with the chosen operating systems.

2.2.2 Lab Architecture Overview

The virtual Lab consists of three virtual machines:

- `OPNsense` The firewall and security gateway, running on FreeBSD.
- `Ubuntu (192.168.1.16)` The victim machine, simulating an internal server.
- `Kali Linux (192.168.1.20)` on internal network / `192.168.176.4` on NAT network, the attacker machine used to simulate both insider threats and external attacks.

To optimize RAM consumption and avoid maintaining a dedicated external attacker VM, a single Kali Linux instance was used with its network adapter switched between **Internal Network** (insider threat simulation) and **NAT Network** (external attacker simulation) depending on the scenario being tested.

2.3 Network Configuration

2.3.1 Network Adapters and Topology

OPNsense was configured with network interfaces:

1. `em0 (WAN)` — Connected to a NAT Network adapter, simulating the Internet-facing interface. OPNsense WAN IP: `192.168.176.7`

2. `em1 (LAN)` — Connected to an Internal Network adapter, serving the internal subnet (`192.168.1.0/24`).OPNsense LAN IP: `192.168.1.1`

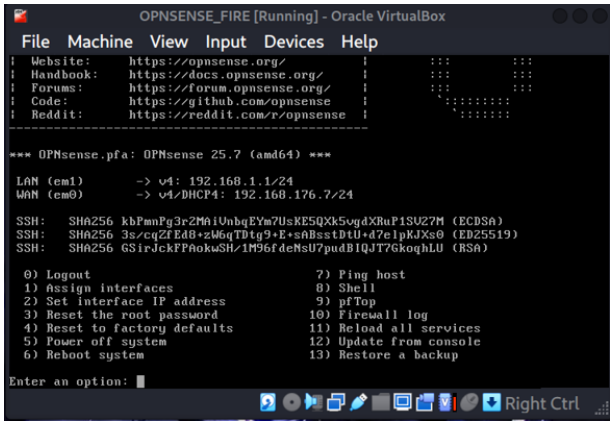


Figure 2.1: OPNsense CLI with interfaces assignments

Ubuntu was connected to the Internal Network (LAN), receiving the IP address `192.168.1.16` via the DHCP service enabled on the LAN interface.

2.3.2 Connectivity Verification

After configuring the interfaces, connectivity was verified using ICMP ping tests from the OPNsense console. OPNsense successfully reached both internal machines, confirming that the LAN routing and DHCP assignments were correctly configured.

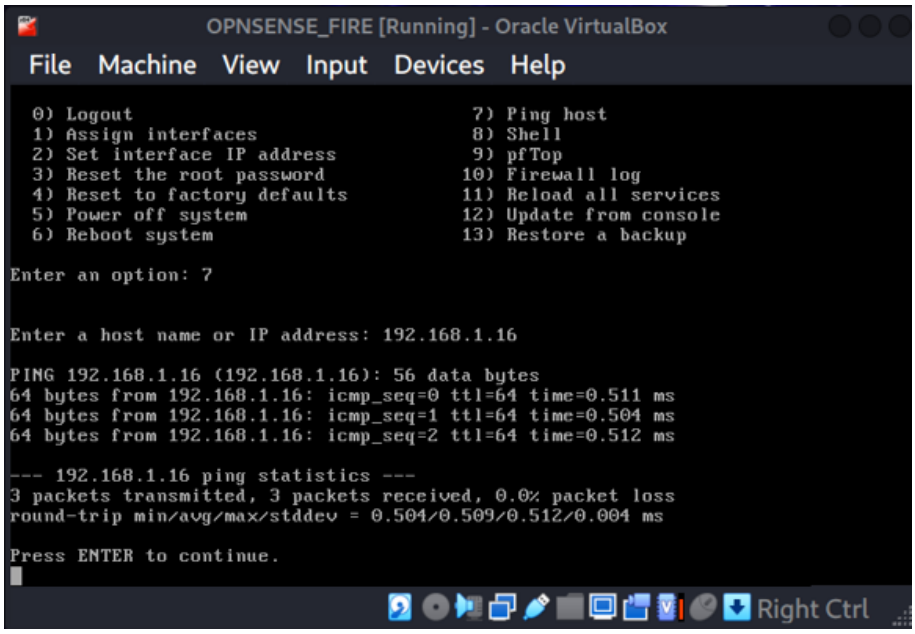


Figure 2.2: OPNsense successfully pinging Ubuntu (192.168.1.16) on the internal LAN

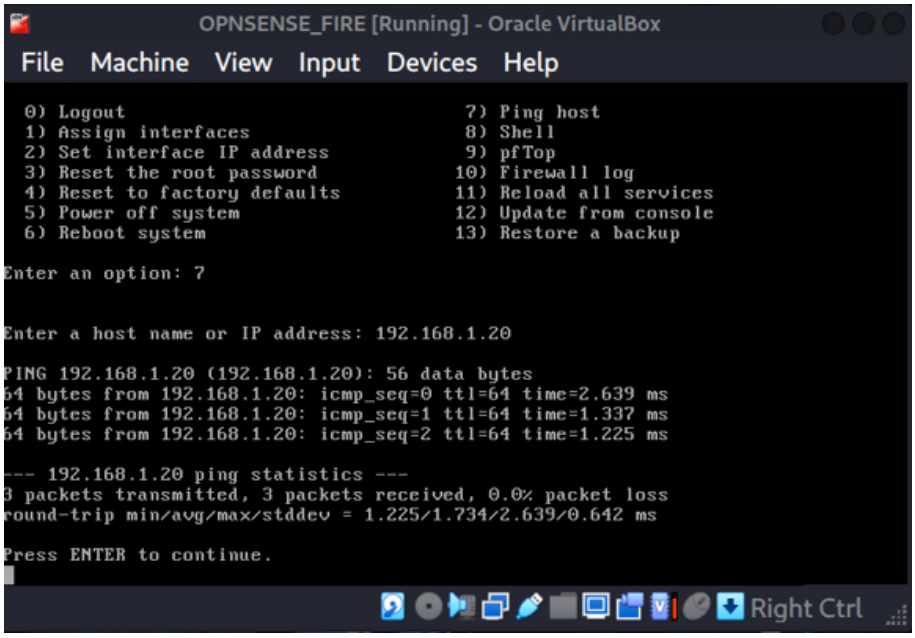


Figure 2.3: OPNsense successfully pinging Kali Linux (192.168.1.20) on the internal LAN

2.3.3 Challenges and Solutions

The initial configuration presented several challenges. Kali Linux placed on the NAT Network could not ping the OPNsense WAN interface, while OPNsense could reach Kali. This was caused by ARP resolution issue specific to the NAT Network mode in VirtualBox. The issue was resolved by ensuring both OPNsense and Kali were attached to the same name NAT Network instance, allowing them to communicate on the same subnet (`192.168.176.0/24`).

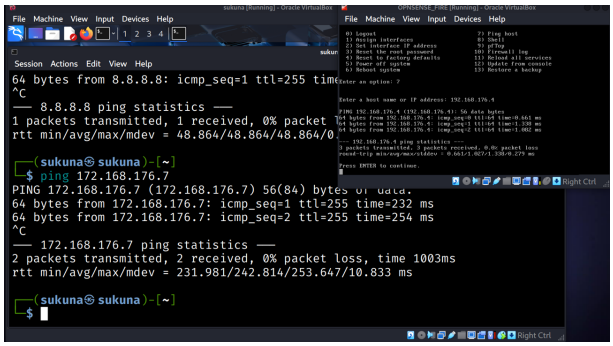


Figure 2.4: Bidirectional connectivity confirmed between Kali Linux and OPNsense WAN

2.4 OPNsense Firewall Configuration

2.4.1 Introduction

OPNsense is a free open-source firewall and routing platform based on FreeBSD. It provides a modern web-based graphical interface for managing firewall rules, NAT, VPN, and integrated security services such as Suricata IDS/IPS. This web GUI is accessible from the LAN side by browsing `http://192.168.1.1`.

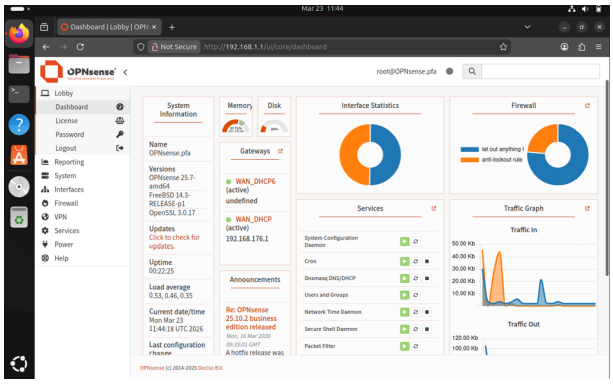


Figure 2.5: OPNsense web GUI accessed from Ubuntu

2.4.2 Interface Configuration and Default Behavior

By default, OPNsense applies a **default deny** policy on the WAN interface, blocking all inbound traffic. This was confirmed by attempting to ping the OPNsense WAN IP from Kali, which failed until an explicit rule for ICMP was added.

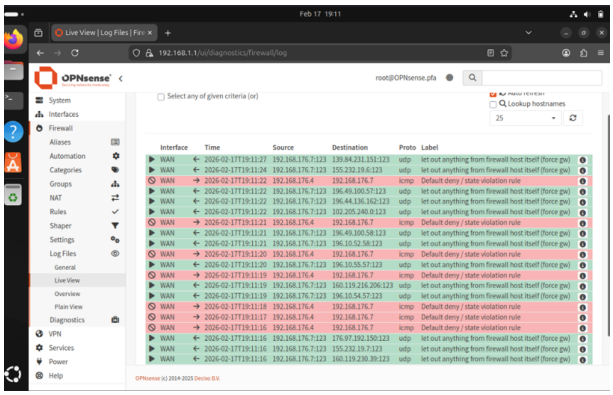


Figure 2.6: OPNsense Live View showing blocked ICMP traffic

Three progressive scenarios were tested to demonstrate how firewall misconfiguration can expose internal services.

Phase 1 : Secure by Default With no port forwarding configured, Nmap scans from Kali (WAN side) reported all ports as `filtered`, confirming that OPNsense blocks all inbound WAN traffic by default.

Phase 2 : Misconfigured WAN Rule A WAN pass rule was added allowing TCP traffic from Kali (192.168.176.4) to Ubuntu (192.168.1.16) on port 22. Despite this rule, Nmap still reported port 22 as `filtered` because the rule allowed traffic to the firewall but did not forward it to the internal host. This demonstrates that firewall rules alone do not expose internal services.

Phase 3 : NAT Port Forward with Pass Association A NAT Port Forward rule was then configured on the WAN interface with the following parameters:

- **Interface:** WAN
- **Protocol:** `TCP`
- **Source:** `192.168.176.4` (kali)
- **Destination:** WAN address
- **Destination Port:** 22 (SSH) and 80 (HTTP)
- **Redirect Target IP:** `192.168.1.16` (Ubuntu)
- **Filter Rule Association:** Pass

With this configuration, Nmap reported port 22 and port 80 as `open`, confirming that the attacker could now reach the victim machine. This demonstrates how a single misconfigured NAT rule can expose internal services to external attackers.

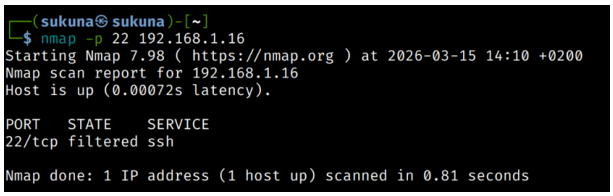


Figure 2.7: Nmap scan showing all ports filtered

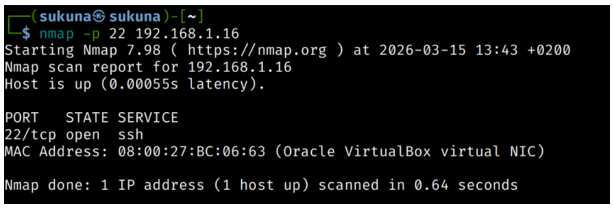


Figure 2.8: Nmap scan report after NAT port forwarding with Pass association

2.4.3 Web Application Attack Without IDS

An Apache2 web server was installed on ubuntu with deliberately vulnerable PHP page (`vuln.php`) that echoed user input without sanitization. From kali, a Cross-Site Scripting (XSS) payload was injected:

```
http://192.168.176.7/vuln.php?name=<script>alert('XSS')</script>
```

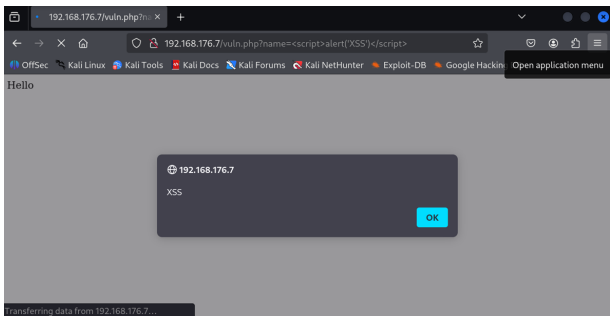


Figure 2.9: XSS Payload successfully executed without IDS detection

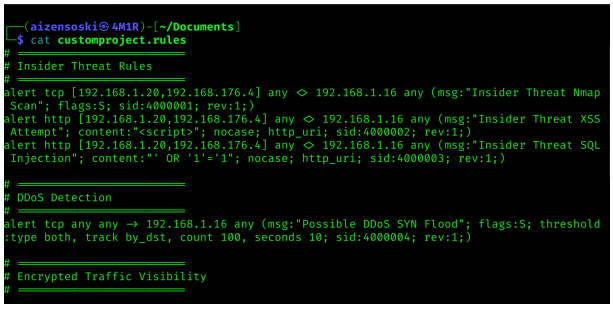
2.5 Suricata IDS Configuration

2.5.1 Introduction

Suricata is a free and open-source network threat detection engine developed by the Open Information Security Foundation (OISF). It supports IDS, IPS, and network security monitoring modes. In this project, Suricata was integrated directly into OPNsense and configured to monitor the LAN interface, enabling detection of both external attacks routed through the firewall and insider threats originating from within the internal network.

2.5.2 Installation and Rule Sets

Suricata was enabled through the OPNsense web interface. The Emerging Threats Open ruleset was downloaded and activated. Custom rules were also written and deployed via FileZilla to `/usr/local/etc/suricata/rules/` to ensure detection of the specific attack scenarios tested in this project.



```
┌──(aizensoski㉿4M1R)-[~/Documents]
└─$ cat customproject.rules
# ═══════════════════════
# Insider Threat Rules
# ═══════════════════════
alert tcp [192.168.1.20,192.168.176.4] any <> 192.168.1.16 any (msg:"Insider Threat Nmap
 Scan"; flags:S; sid:4000001; rev:1;)
alert http [192.168.1.20,192.168.176.4] any <> 192.168.1.16 any (msg:"Insider Threat XSS
 Attempt"; content:"<script>"; nocase; http_uri; sid:4000002; rev:1;)
alert http [192.168.1.20,192.168.176.4] any <> 192.168.1.16 any (msg:"Insider Threat SQL
 Injection"; content:"' OR '1'='1"; nocase; http_uri; sid:4000003; rev:1;)

# ═══════════════════════
# DDoS Detection
# ═══════════════════════
alert tcp any any -> 192.168.1.16 any (msg:"Possible DDoS SYN Flood"; flags:S; threshold
:type both, track by_dst, count 100, seconds 10; sid:4000004; rev:1;)

# ═══════════════════════
# Encrypted Traffic Visibility
# ═══════════════════════
```

Figure 2.10: Custom Rules written for each attack

2.5.3 Challenges and Solutions

Several issues were encountered during Suricata configuration. Initially, no alerts appeared despite attacks being executed.Two issues were identified: the default Emerging Threats ruleset did not match the lab payloads, and Suricata was bound to the wrong interface. These were resolved by writing custom rules and correctly binding Suricata to the LAN interface (`em1`).

2.6 Attack Simulation

2.6.1 Attack Scenarios and Methodology

With Suricata enabled, the following attack scenarios were executed from Kali Linux and verified through Suricata alerts:

- **XSS Attack (Insider):** The same XSS payload sent from kali on the internal network was detected by Suricata with the alert *"Insider Threat XSS Attempt"*
- **Port Scanning (Nmap):** An `nmap -sS` scan was detected as `''Insider Threat Nmap Scan''`.
- **DoS Attack:** HTTP flooding using Apache Benchmark (`ab`) caused the *"DoS HTTP Flood"* alert.
- **DDoS/SYN Flood:** `hping3 -S -flood` generated a *"Possible DDoS SYN Flood"* alert.
- **Encrypted Traffic:** A TLS connection attempt was flagged as *"Suspicious Encrypted Traffic Detected"*.
- **CVE-2024-20353 Simulation:** HTTP requests to `/webui` and `/vpn` paths triggered alerts for *"CVE-2024-20353 Cisco IOS XE Exploit Attempt"*.

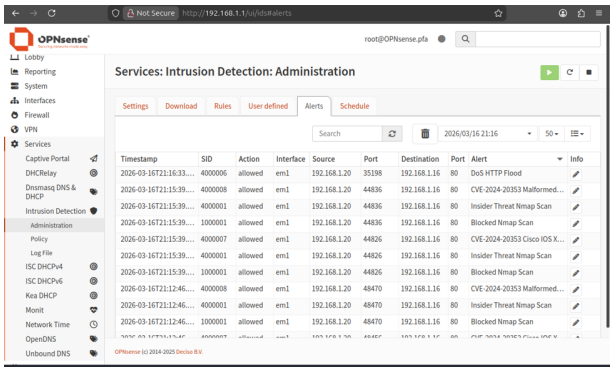


Figure 2.11: Suricata alert dashboard

2.7 Dataset Preparation

2.7.1 Log Extraction

Suricata logs all detected events in EVE JSON format (`eve.json`). After completing the attack simulation session, the log file was transferred from OPNsense to the host machine using `Filezilla` and SCP:

```
scp root@192.168.1.1:/var/log/suricata/eve.json ~/eve.json
```

The dataset collected contained 35,705 alert entries.

2.7.2 Feature Selection and Preprocessing

Each log entry contains structured fields including source IP, destination IP,port numbers, protocol, alert signature, and timestamp. These fields were extracted as features for the machine learning model. The dataset was labeled: entries matching attack signatures were labeled as **malicious (1)**, while normal traffic entries were labeled as **benign (0)**.

Before applying the machine learning model, the Suricata log data was prepared through several preprocessing steps. Missing values were handled, categorical attributes such as protocol type and alert signatures were converted into numerical representations, and numerical features were normalized. The dataset was also examined for class imbalance between benign and malicious traffic in order to improve the reliability of the model during training.

2.8 Conclusion

This chapter presented the implementation of the virtual security lab, including the configuration of the OPNsense firewall, the deployment of Suricata IDS/IPS, and the simulation of different attack scenarios. The experiments demonstrated that while the firewall effectively controls network traffic, it cannot detect certain application-layer attacks such as XSS. The alerts and logs generated by Suricata provide valuable data that will be analyzed in the next chapter through simulations and experimentaal results.

Bibliography

[1] J. Zhang, M. Zulkernine, and A. Haque, "Random-forests-based network intrusion detection systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 38, no. 5, pp. 649–659, 2008.

[2] Microsoft, "What is network security?," 2024.

[3] H. Academy, "Network security foundations," 2024. Accessed: March 2026.

[4] Fortinet, "Network security threats," 2025. Accessed: March 2026.

[5] CloudFlare, "whatismalware," 2026. Accessed: March 2026.

[6] CloudFlare, "whatisddos," 2026. Accessed: March 2026.

[7] CloudFlare, "whatissyn," 2026. Accessed: March 2026.

[8] CiscoSystem, "Firewalls," 2026. Accessed: March 2026.

[9] malekal, "Firewalls," 2026. Accessed: March 2026.

[10] geeksforgeeks, "Firewalls," 2026. Accessed: March 2026.

[11] checkpoint, "Firewalls vulnerabilities," 2026. Accessed: March 2026.

[12] A. Hozouri, A. Mirzaei, and M. Effatparvar, "A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges," *Discover Artificial Intelligence*, vol. 5, no. 314, 2025.

[13] Fortinet, "Qu'est-ce qu'un système de détection d'intrusion (ids)?," 2025.

[14] GeeksforGeeks, "Intrusion detection system (ids)," 2025.

[15] C.-Y. Ho, Y.-C. Lai, I.-W. Chen, F.-Y. Wang, and W.-H. Tai, "Statistical analysis of false positives and false negatives from real traffic with intrusion detection/prevention systems," *IEEE Communications Magazine*, vol. 50, no. 3, pp. 146–153, 2012.

[16] M. M. Issa, M. Aljanabi, and H. M. Muhialdeen, "Subject review: Intrusion detection system (ids) and intrusion prevention system (ips)," *Global Journal of Engineering and Technology Advances*, vol. 14, no. 2, pp. 155–158, 2023.

[17] I. H. Sarker, "Multi-aspects ai-based modeling and adversarial learning for cybersecurity intelligence and robustness: A comprehensive overview," *Security and Privacy*, vol. 6, no. 5, p. e295, 2023.

[18] A. Pinto, L.-C. Herrera, Y. Donoso, and J. A. Gutierrez, "Survey on intrusion detection systems based on machine learning techniques for the protection of critical infrastructure," *Sensors*, vol. 23, no. 5, p. 2415, 2023.